Gemmini with 2:4 and 4:8 Sparsity: Streaming Benchmark Results

Run **experiments/streaming/runs/20261109-144252** · 52 result rows, 52 verified · area from analysis/area/area.csv · small-scale result from verification/runs/fair-20261103-151404

What was measured

Each test program multiplies one real-size layer of a neural network: C = A × W with 8-bit inputs and 32-bit outputs. A and W start in main memory and are streamed into the accelerator while it works. The clock-cycle count runs from the first load to the last result being written back. The chip is simulated exactly, one clock tick at a time (Verilator RTL simulation of the Chipyard SoC). Three chips were compared: Berkeley's unmodified Gemmini (the **stock** chip) and our **2:4** and **4:8** versions. Each ran up to three programs on the same layer:

- **Berkeley's tiler** (tiled_matmul_auto, using Gemmini's built-in hardware loop): Berkeley's best way to run a dense layer, on the stock chip. It is the main reference.
- **Our hand tiler, dense**: the streaming loop written for this benchmark, with ordinary (uncompressed) weights.
- **Our hand tiler, sparse**: the same loop with half-size compressed weights plus the index tiles that say where each kept weight belongs (2:4 or 4:8).

Every variant computes the same answer from the same pruned weights. The dense and sparse programs of one pattern have byte-identical code; only two data flags differ. Each run is preceded by an untimed warm-up. The output and the accumulators are filled with a poison value first, and the caches are flushed by reading a 2 MiB buffer, so every run starts cold. After each run the CPU hashes the whole output and compares 8 sampled rows in full, and the host checks the hash again against an independent numpy reference. **All 52 result rows are correct.**

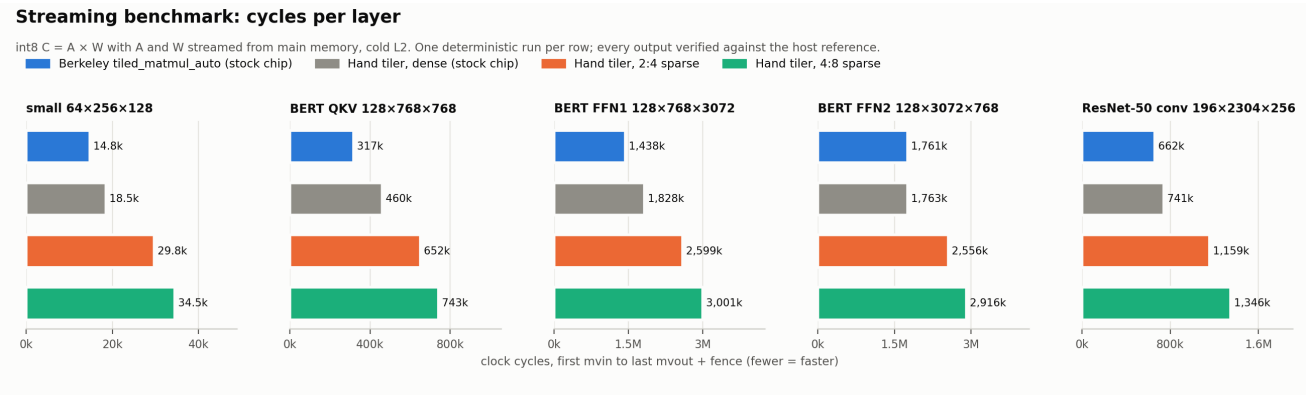
Every number is one deterministic run. Totals are unchanged when all non-reset register and SRAM contents are randomised (simulator seeds 1–3, verified), so no ranges over seeds are given.

Results

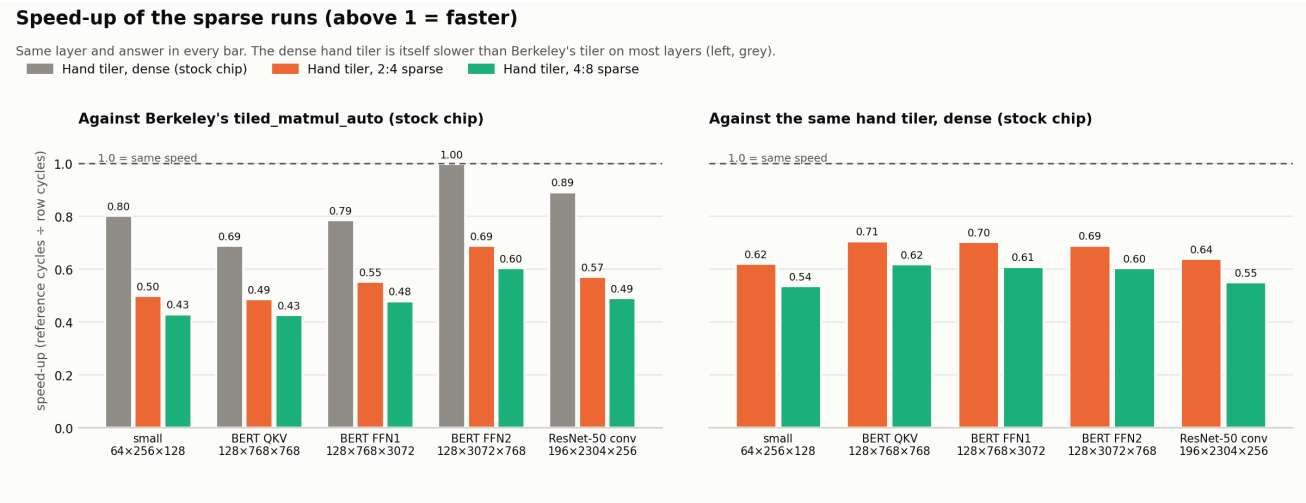
Layer	Berkeley's tiler (stock)	Hand tiler, dense (stock)	Hand tiler, 2:4 sparse	Hand tiler, 4:8 sparse
small (64×256×128)	14,840	18,487 0.80× vs Berkeley	29,777 0.50× vs Berkeley 0.62× vs dense tiler	34,518 0.43× vs Berkeley 0.54× vs dense tiler
BERT QKV (128×768×768)	316,791	460,206 0.69× vs Berkeley	651,640 0.49× vs Berkeley 0.71× vs dense tiler	743,192 0.43× vs Berkeley 0.62× vs dense tiler
BERT FFN1 (128×768×3072)	1,438,120	1,828,210 0.79× vs Berkeley	2,598,704 0.55× vs Berkeley 0.70× vs dense tiler	3,000,588 0.48× vs Berkeley 0.61× vs dense tiler

BERT FFN2 (128×3072×768)	1,761,377	1,763,352 1.00× vs Berkeley	2,556,093 0.69× vs Berkeley 0.69× vs dense tiler	2,915,792 0.60× vs Berkeley 0.60× vs dense tiler
ResNet-50 conv (196×2304×256)	661,605	741,495 0.89× vs Berkeley	1,158,534 0.57× vs Berkeley 0.64× vs dense tiler	1,346,269 0.49× vs Berkeley 0.55× vs dense tiler

Clock cycles, fewer = faster. Speed-up = reference cycles ÷ this run's cycles, so above 1× is faster. The full table, with every chip and counter, is in stream_table.csv and tables.md.



Cycles per layer. Each panel has its own scale.



Speed-up of the sparse runs against Berkeley's tiler (left) and against the same hand tiler with dense weights (right).

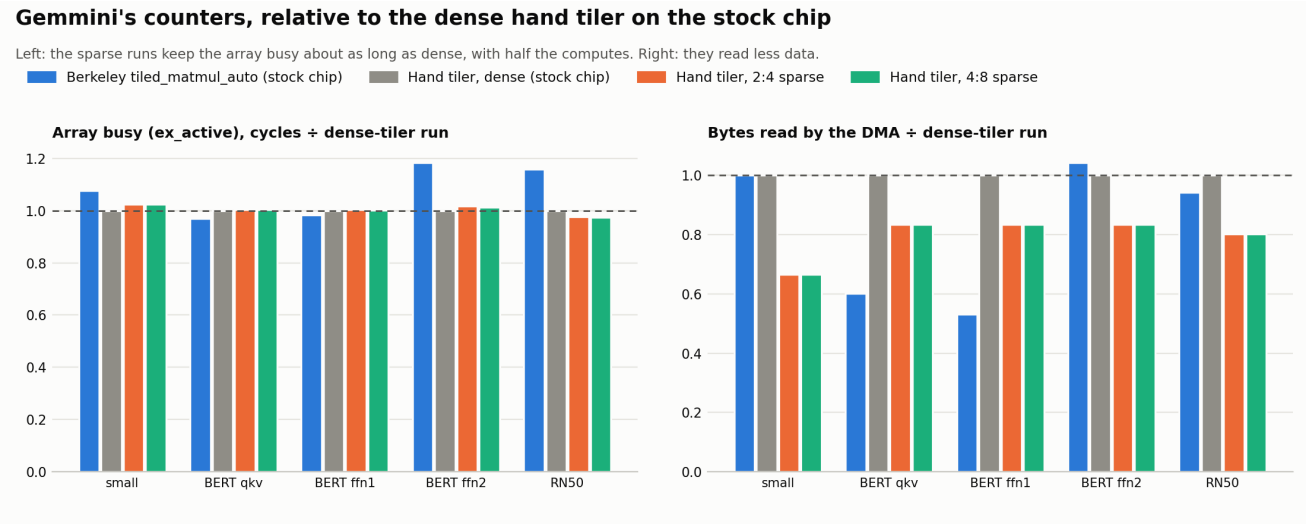
In plain words:

- **Sparse is slower than dense on every layer, on both references.** On the four real layers, 2:4 sparse runs at 0.49×–0.69× of Berkeley's speed and 4:8 at 0.43×–0.60×. Against the same hand tiler with dense weights, 2:4 runs at 0.64×–0.71× and 4:8 at 0.55×–0.62×.
- **4:8 is slower than 2:4 on every layer.**
- **Our dense hand tiler is itself slower than Berkeley's tiler**: 0.80× (small), 0.69× (qkv), 0.79× (ffn1), 1.00× (ffn2), 0.89× (rn50). Part of the sparse runs' gap to Berkeley is therefore the tiler, not sparsity. That is why both references are shown.

- **The extra hardware doesn't slow dense work.** On the 2:4 chip the dense hand tiler takes within 0.8% of the stock chip's cycles on every layer. Berkeley's tiler on the 2:4 and 4:8 chips (run for the small and edge layers) is within 1.6% of the stock chip.

What the counters show

- **The array is busy about as long in sparse mode as in dense mode.** On the real layers, the sparse runs' ex_active (cycles the execute unit spends in its compute state) is within -2.4% to +1.7% of the dense hand tiler's, although sparse mode issues half as many computes.
- **The sparse runs read less data**: 0.80–0.83× the bytes of the dense hand tiler (2:4 and 4:8 read the same amount).
- **Why the sparse runs don't finish sooner (finding of a separate investigation).** The sparse path is limited by the CPU issuing commands, not by Gemmini. In sparse mode Gemmini's hardware loop (LOOP_WS) is not available, so the CPU issues every command itself, and each index tile also costs the CPU several RoCC commands. In the investigation's own counter runs (2:4 chip, small layer, a build with an extra counter set; not part of this run's results.csv), Gemmini's command buffer was empty 37% of the time with this hand tiler and 42% with a leaner rewrite of it, against 8% for Berkeley's tiler: Gemmini often sits idle waiting for the CPU. Details: experiments/streaming/worklogs/investigate.md, sections 9 and 17.
- **Note on the rs_full counter.** Despite its name, Gemmini's rs_full counter does not measure a full command queue. It counts every cycle in which no new command is accepted, idle cycles included, so it is not plotted. Its values are kept in tables.md and stream_table.csv under that meaning.



Gemmini's counters relative to the dense hand tiler on the stock chip: array busy time (left) and bytes read (right).

Chip area

Block	Stock (μm²)	2:4 (μm²)	4:8 (μm²)
Whole accelerator	7,794,492	8,201,671 (+5.2%)	8,546,004 (+9.6%)
Execute unit (array, index FIFO, control)	2,795,913	3,206,114 (+14.7%)	3,549,807 (+27.0%)
Array + input/output skew registers	2,674,665	2,977,755 (+11.3%)	3,259,037 (+21.8%)
16×16 array	2,443,103	2,707,036 (+10.8%)	2,975,774 (+21.8%)
One processing element	7,866	8,357 (+6.2%)	8,736 (+11.1%)

Standard-cell area from a Yosys synthesis to the open SkyWater 130 nm high-density cells (sky130_fd_sc_hd, typical corner), before place and route. The SRAM macros are left out because they are identical in all three chips. This compares the chips; it is not a die size.

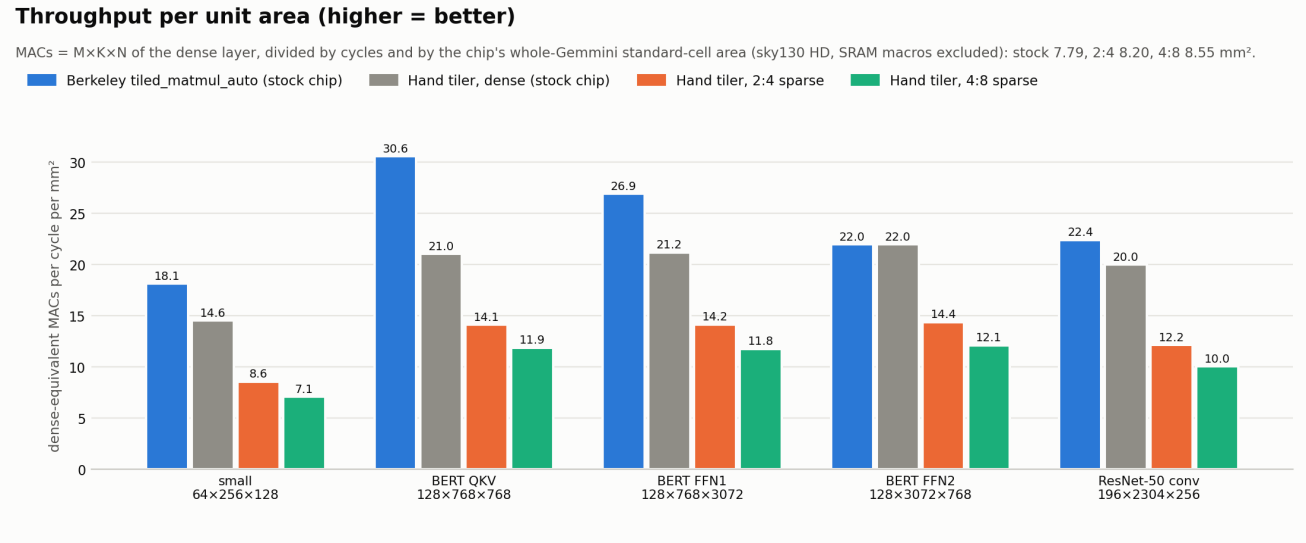
The whole accelerator grows by 5.2% for 2:4 and 9.6% for 4:8. The array and its skew registers account for 74% (2:4) and 78% (4:8) of that growth.

Throughput per unit area

Layer	Berkeley's tiler (stock)	Hand tiler, dense (stock)	Hand tiler, 2:4	Hand tiler, 4:8
small (64×256×128)	18.1	14.6 0.80× vs Berkeley	8.6 0.47× vs Berkeley	7.1 0.39× vs Berkeley
BERT QKV (128×768×768)	30.6	21.0 0.69× vs Berkeley	14.1 0.46× vs Berkeley	11.9 0.39× vs Berkeley
BERT FFN1 (128×768×3072)	26.9	21.2 0.79× vs Berkeley	14.2 0.53× vs Berkeley	11.8 0.44× vs Berkeley
BERT FFN2 (128×3072×768)	22.0	22.0 1.00× vs Berkeley	14.4 0.65× vs Berkeley	12.1 0.55× vs Berkeley
ResNet-50 conv (196×2304×256)	22.4	20.0 0.89× vs Berkeley	12.2 0.54× vs Berkeley	10.0 0.45× vs Berkeley

Dense-equivalent multiply-accumulates (M×K×N of the layer) per clock cycle per mm² of whole-accelerator standard-cell area: stock 7.79 mm², 2:4 8.20 mm², 4:8 8.55 mm². For the sparse rows, half of those multiply-accumulates are skipped zeros.

Per unit area, 2:4 sparse delivers 0.46×–0.65× and 4:8 0.39×–0.55× of Berkeley's throughput on the real layers. That is lower than the speed ratios above, because the sparse chips are also larger.



Throughput per mm², every layer and variant.

Small-scale result: the fair benchmark (64×128×32)

Clock cycles	Berkeley original	Ours, 2:4	Ours, 4:8
Array time, weights reused	1,260	672 (1.88×)	672 (1.88×)
Whole job, weights reused	1,541	1,395 (1.10×)	1,607 (0.96×)
Whole job, fresh weights	1,555	2,281 (0.68×)	3,170 (0.49×)
Whole job, fresh weights, indexes batched 4 tiles ahead	–	2,110 (0.74×)	2,964 (0.52×)

verification/fair_perf.c: one small matrix multiply (64×128×32), the same program on every chip. Each number is one deterministic run (seeds 1, 2, 3 identical). The ratio is Berkeley's cycles ÷ ours.

With the weights reused, sparsity halves the array's working time (1.88× for both patterns). The whole job is 1.10× for 2:4 and 0.96× for 4:8. With fresh weights (and index tiles) for every step, both are slower than Berkeley's: 0.68× and 0.49×. Berkeley's exact program with sparsity off takes 1,541 and 1,555 cycles on our 2:4 chip, against Berkeley's 1,541 and 1,555.

Caveats

- **The hand tiler is not as good as Berkeley's.** On dense weights it reaches 0.80× (small), 0.69× (qkv), 0.79× (ffn1), 1.00× (ffn2), 0.89× (rn50) of Berkeley's tiled_matmul_auto. Sparse results are therefore reported against both references.
- **Counter details.** The bytes-read counter adds the whole 64 B request size on every 16 B beat, so it reads 4× the real bytes; the numbers here are divided by 4. Read through a counter snapshot, it returns 0, so it was read live. loop_matmul_active means something only for Berkeley's tiler. ex_active includes stalls inside the compute state. rs_full (tables only) counts cycles with no command accepted, idle included; its window starts at its own configuration and ends at its own read, so it is slightly longer than the timed region and on the smallest layers its share of the total can reach 101%.

- **Area is pre-layout standard-cell area** (Yosys 0.69, sky130 HD typical corner; the ABC mapping step &fraig -x was left out, the same for all chips, because it doesn't finish on one of Berkeley's address-generator modules). There are no wires, no SRAM macros, and no clock-speed or power numbers. The extra multiplexer in front of every multiplier may lower the maximum clock speed; that was not measured.
- **Coverage.** Berkeley's tiler was run on the 2:4 and 4:8 chips only for the small and edge layers. The 4:8 chip's dense hand-tiler run exists only for those layers too, so the dense reference is always the stock chip's run. Five of the large-layer runs were simulated on a second, validated machine (its simulators were built from the same sources; results.csv records the host and simulator hash of every row). The edge layer (36×128×64) checks the partial-tile path and appears only in the tables.
- **Simulation, synthetic data.** The weights are random and pruned; AI accuracy was not measured. Simulation gives exact cycle counts, not clock speed or power.

Where the numbers come from

experiments/streaming/scripts/collect.py writes results.csv from the simulation logs and checks every output. experiments/streaming/analysis/stream_analysis.py computes every ratio, the per-area throughput and the figures from results.csv, analysis/area/area.csv (analysis/area.py) and the fair run, read through verification/nmdata.py. This document is generated by build_stream_paper.py from those outputs. No number was typed by hand.